Human Activity Recognition Using Smartphones

1 - Introduction

Human Activity Recognition (HAR) is a research field that seeks to automate interpretability for a continuous classification challenge: mapping physical human data into discrete classes. Empirically, the application of HAR has engaged fields of healthcare (patient monitoring), sports (activity tracking), security (monitoring), and human-computer interaction (smart homes, user interfaces) (Ravuri, 2024).

2 - Dataset

The dataset consists of data captured using the embedded Micro-Electro-Mechanical System (MEMS) of Samsung Galaxy S II (Anguita, 2013). This data was collected from the accelerometer and the gyroscope of the device. Empirically, these two grouped sensory inputs have been pivotal to the development of context-based UI in the mobile application field (Caro-Alvaro et al., 2024). Six different activities of daily life (ADLs) were recorded across 30 subjects. These activities include: *sitting, standing, laying down, walking, walking downstairs,* and *walking upstairs.* Data was sampled at a frequency of 50 Hz sampling rate.

The data has been preprocessed using a median filter, which removes irregular spikes and outliers (Anguita et al., 2013). A Butterworth filter at 20 Hz has also stripped most high-frequency electronic or mechanical vibration noise that could be present in such recordings. Gravity also has to be accounted for with such a low-pass filter, where 0.3 Hz was found by Anguita et al. (2013) to be an optional corner frequency. Continuous time signals have had to be divided into fixed-width slices of 2.56 seconds with a 50% overlap, and at 50 Hz this window contains exactly 128 readings, which is optimized for power-of-two vectors. This amounts to a total of 17 signals obtained. The features are also divided into time-domain and frequency-domain features. The time-domain features (time-domain) are most useful for the models to categorize different stationary activities like laying, sitting, and standing, because it tells more about the orientation relative to gravity. The frequency-domain features (frequency-domain) are more useful for distinguishing walking, walking upstairs, and downstairs because they have similar acceleration but more unique frequency and cyclic energy (Cruciani et al., 2018). The sample set exhibits a balanced class distribution across subjects, confirmed by a 0.9965 normalized Shannon Entropy (Shannon, 1948).

3 - Related Work

Tishby and Zaslavsky (2015) argue that data derived from sensors commonly used in HAR systems introduce too much of a magnitude of dimensionality, which supports a need for a more interpretable aggregation of the sensor data. Huang et al. (2024) address Tishby and Zaslavsky's concerns on "the curse of dimensionality" by proposing a post-hoc interpreter that tests counterfactual perturbations competitively. Given our lack of experience with analysis of sensory data, a comprehensive research overview had to be undertaken to define the model in the context of domain knowledge rules. Another paper concerning a similar problem statement with a similar approach using XGBoost has also been of huge interest in the research process (Zhang et al., 2019).

4 - Feature Engineering

To prevent an overfit model from an overextension of features, numerous feature extraction and engineering practices had to be applied to the dataset.

Stage	Operation	Feature count
0	Initial dataset.	561
1	Duplicate column removal.	540
2	Removing redundant frequency-domain features.	488
3	Safe variance thresholding.*	451
4	Correlation filter.*	219
5	Yeo-Johnson Transform	219
6	Principal Component Analysis*	88

*** :** Considerations for data leakage implemented, which is explained in Chapter 5.

Stage 1:
Features that contain an identical set of samples are removed while keeping the first, as duplicate samples do not contribute to the model's interpretation.

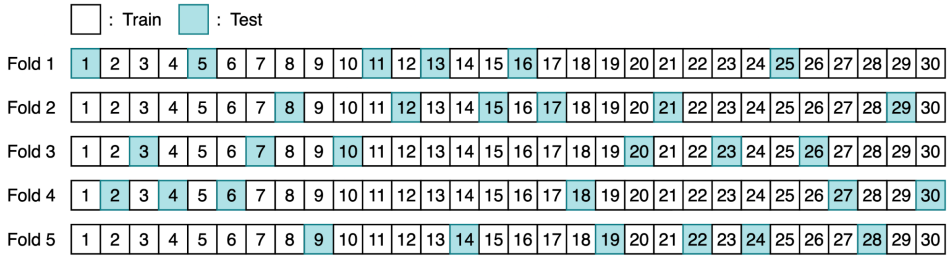
Stage 2:
As all the features have been projected into two mathematical spaces, there is no inherent necessity for some features to be represented twice. There are instances where a value from a time-domain value yields different information from the frequency-domain counterpart. While this discrepancy has the opportunity to add relevant dimensionality to the model, it can contribute to the phenomena "curse of dimensionality". A strong prominence of this can occur when looking at the projected standard deviation from values comprising acceleration. With the features: '*tBodyAcc-std()-X'* and *'fBodyAcc-std()-X'*, the latter frequency-domain value may have a higher value because of a sharp increase in acceleration during its captured time frame; the signals are clustered, rather than capturing an essential instance of movement, making the feature susceptible to noise.

Stage 3:
Within the domain of sensory data, removing features solely based on whether the sample set falls within a particular variance threshold may reduce the data's interpretability. A feature may respond subtly to specific activities, making the sample value pool non-variant but still relevant to the model's interpretability. This is why this particular reduction step needs to be examined carefully. A subtle variance threshold removal was done for all features with > 1% variance, producing the removal of 37 features.

Stage 4:
With the implementation of a correlation filter with a safe threshold of 95%, we prevent multicollinearity. Although the algorithms of C5.0, CART, and XGBoost utilize several methods of preventing multicollinearity by essentially pruning features depending on impurity or other performance metrics across the staged features, keeping the features anyway makes the models less interpretable, as the feature importance remains more inconsistent across model permutations (Kuhn & Johnson, 2013, pp. 47–51).

5 - Modelling Pipeline

A consistent monitoring of the integrity of the data had to be undertaken during the feature engineering of a selection of operations (stage 4 and 6) to avoid data leakage (Ambroise & McLachlan, 2002). The subject-safe cross-validation module *GroupKFold* was implemented (Pedregosa et al., 2011). With the established group distribution, the grouping of subjects within each of these folds was picked at random, as neither permutation proves to create class imbalances. Another rationale for the intra-subject distinction between train and test sample sets is the aforementioned 2.56-second overlap transformation mentioned in Chapter 2, which, without caution, would lead to data leakage within the same subject.



6 - XGBoost

XGBoost, at its core, exhibits a potentially greater interpretability for sensory HAR data even without feature engineering, compared to other models. This is supported by its mechanisms of extensive gradient boosting of the competing decision trees and its continuous regularization, which addresses concerns of high dimensionality and the presence of sensory noise (Zhang et al., 2019).

Hyperparameters

By looking at the data from experiments with different XGBoost hyperparameter permutations, we have observed that the parameters that greatly determine the shape of the trees have the biggest impact on the model results. These include: *n_estimators* (number of trees) and *learning_rate* (the weight of each tree). Both of these variables can be described as having a strong inverse relationship; a lower learning rate requires a higher number of estimators.

Several mechanisms implemented in XGBoost aim to contribute to the model generalizing rather than only memorizing the training data (overfitting). Naturally, this is especially interesting to us, given the complexity of the sample set and the complexity XGBoost offers (compared to other classification models), combined with the assumed overfitting results (more on that in chapter); what became most interesting to us were the elements that aim to avert overfitting. One of these countermeasures includes "*early stopping"*. In the official XGBoost documentation, the phrase "*early stopping*" is attributed to the mechanism that ought to stop the training of the model when the ensemble of trees does not contribute to a lower validation error by comparing it to a selected *validation set*, independent of the *test set* (XGBoost Contributors, 2026).

Though there are other functions that stop the algorithm early, such as in the case of when the algorithm is in its pre-pruning phase, when the model is building trees. During this phase, its sequential course is determined by a maximum/minimum value defined by *max_depth* and/or *min_child_weight*. The algorithm reinspects each tree by inspecting each of its splits. If the split does not satisfy the minimum loss reduction defined by the parameter *gamma*, it will stop early and prune the selected node's leaves. All of the aforementioned parameters are essential to controlling the model's output and avoiding overfitting.

7 - CART (Classification and Regression Trees)

We also implemented a CART model in this project. It constructs a decision tree by splitting the dataset and evaluating the best split per feature. The model then chooses the optimal split, and each node gets two child nodes (GeeksForGeeks, 2025). This was done recursively until one of the stopping conditions was met. The tree-based structure of this model makes the decisions easily observable when plotted out in a tree graph.

Hyperparameters

The hyperparameter optimization was implemented by using an inner group-based split to guarantee no overlap during tuning. Each model was then evaluated based on its macro F1-score. The configuration with the highest score was selected and refit on all the training data and then evaluated on the final test set.

There were four hyperparameters that were tuned for this model. Criterion, maximum depth, minimum samples per split, and minimum samples per leaf.

The criterion is the function that evaluates which feature and threshold produce the best split for a node. It is an impurity criterion divided into *Gini impurity* and *entropy*.

Gini impurity is somewhat biased toward dominant classes, whereas entropy is sensitive to small fluctuations but can give more balanced splits.

8 - C5.0

C5.0 is a supervised decision-tree algorithm, which, like CART, recursively splits the data into subsets by selecting certain features and values that improve class separation. C5.0, instead of using an *impurity criterion*, uses the metric named *gain ratio*. This method favors splits that reduce uncertainty about class labels. The model then represents the classifiers as a decision tree.

Hyperparameters

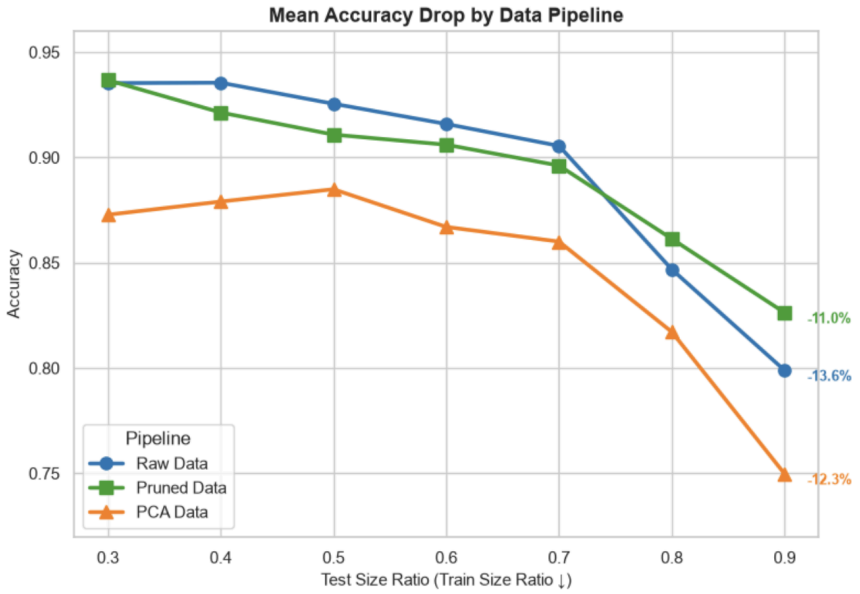
C5.0 has minimum samples per leaf as a hyperparameter, like CART. In addition to this, we could adjust the global pruning and winnowing. Winnowing is a feature selection step in the C5.0 model, but our model actually performed more optimally without it. This is likely due to our extensive feature extraction step.

9 - Results

XGBoost

The parameter grid of each permutation of XGBoost instances has been defined by continuously inspecting the results, experimentally tweaking the parameters, and using found permutations from RandomizedSearchCV.

Three sample sets encompassing different feature engineering permutations were tested during modelling: Dataset 1 (ran through stage 0-1), Dataset 2 (stage 0-4), and Dataset 3 (stage 0-6).



The graph above shows the discrepancy between sample set permutations with GroupShuffleSplit to illustrate how the model responds to differing data amounts (Pedregosa et al., 2011). Suspicions of overfitting were present, but after it was known that, despite increasing the test size, the predictions remained accurate and stable in some ranges, it changed the narrative. The predictions posed a > 75~ % across macro F1-, recall- (sensitivity) and accuracy scores, despite the test size. PCA proved to provide less interpretability compared to the others, given that the accuracy scoring actually increased despite decreasing the test size.
If we can rule out data leakage, these are exceptional results. Another key finding from this graph is how the relationship between Dataset 1 and Dataset 2 can be looked at as an example of the bias-variance tradeoff. We see that Dataset 1 (the least feature-engineered permutation) sequentially starts to suffer from poorer accuracy results, in line with the increasing abundance of available training data more than the pruned sample set.

CART

The CART model used macro-F1 as its hyperparameter selection metric. The hyperparameter grid configuration was refitted on the complete training partition and separately evaluated. The hyperparameter combination that yielded the best result for the CART model both with and without the PCA dataset was when the minimum samples per leaf was 10. This acts as a regularizer that prevents the tree from memorizing individual records, while still keeping a deep hierarchical logic. This allowed the max depth to be set to None with less risk of overfitting, as the amount of minimum leaves would even it out. For the non-PCA-trained model, the hyperparameters that yielded the best results used the Gini impurity method, unrestricted maximum depth, minimum samples per split were 2, and minimum number of samples per leaf was 10. This build supports generalization and can help prevent overfitting, as an average accuracy of 85.30% on the held-out test data indicates that the tree was able to split down to groups of at least 10 samples, avoiding being confused by noise. The best CART model achieved a test accuracy of 85.30% and a macro-F1 of 0.85.

C5.0

This model's hyperparameters were also chosen based on optimal macro-F1 scores.
For the C5.0 model trained on the full dataset, the best hyperparameters included no global pruning or winnowing, keeping all 217 features. This model performed better with a little less regularization than the CART model, using a minimal number of 5 samples per leaf.

This indicates that the model gained a benefit from somewhat more detailed rules, but did not need to create classifications narrowed down to single training records. The best C5.0 configuration got a final test accuracy of 84.34% and a macro-F1 of 0.84.

7 - Conclusion & Reflection

Evaluating the results from the HAR data through the models has been an interesting dive into how these models can capture complex relationships between data features. Across experiments, applying PCA transformations offered no performance benefit across the models. These results align with the nature of the selected models. PCA contributes to creating a data space where global geometric distance is relevant, making the transformation beneficial for regressional models where linear relationships benefit the accuracy of the model. Despite this, domain-aware pruning proved to be superior by a larger factor, contributing to maintaining a strong model generalization across train/test permutations.

Literature list

Ambroise, C., & McLachlan, G. J. (2002). Selection bias in gene extraction on the basis of microarray gene-expression data. Proceedings of the National Academy of Sciences, 99(10), 6562–6566. https://doi.org/10.1073/pnas.102102699

Anguita, D., Ghio, A., Oneto, L., Parra, X., & Reyes-Ortiz, J. L. (2013). A public domain dataset for human activity recognition using smartphones. The European Symposium on Artificial Neural Networks. https://api.semanticscholar.org/CorpusID:6975432

Anguita, J. R. D. (2013). Human activity recognition using smartphones [Dataset]. UC Irvine Machine Learning Repository. https://doi.org/10.24432/c54s4k

Caro-Alvaro, S., Garcia-Lopez, E., Brun-Guajardo, A., Garcia-Cabot, A., & Mavri, A. (2024). Gesture-based interactions: Integrating accelerometer and gyroscope sensors in the use of mobile apps. Sensors, 24(3), Article 1004. https://doi.org/10.3390/s24031004

Cruciani, F., Cleland, I., Nugent, C., McCullagh, P., Synnes, K., & Hallberg, J. (2018). Automatic annotation for human activity recognition in free living using a smartphone. Sensors, 18(7), Article 2203. https://doi.org/10.3390/s18072203

Del Rosario, M., Redmond, S., & Lovell, N. (2015). Tracking the evolution of smartphone sensing for monitoring human movement. Sensors, 15(8), 18901–18933. https://doi.org/10.3390/s150818901

GeeksforGeeks. (2025, December 4). CART (Classification and Regression Tree) in machine learning. https://www.geeksforgeeks.org/machine-learning/cart-classification-and-regression-tree-in-machine-learning/

Huang, Y., Zhou, Y., Zhao, H., Riedel, T., & Beigl, M. (2024). Explainable deep learning framework for human activity recognition (arXiv:2408.11552). arXiv. https://doi.org/10.48550/arxiv.2408.11552

Kuhn, M., & Johnson, K. (2013). Applied predictive modeling. Springer.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

Ravuri, A. (2024). A systematic literature review on human activity recognition. Journal of Electrical Systems, 20(6s), 1175–1191. https://doi.org/10.52783/jes.2848

Shannon, C. E. (1948). A mathematical theory of communication. Bell System Technical Journal, 27(3), 379–423. https://doi.org/10.1002/j.1538-7305.1948.tb01338.x

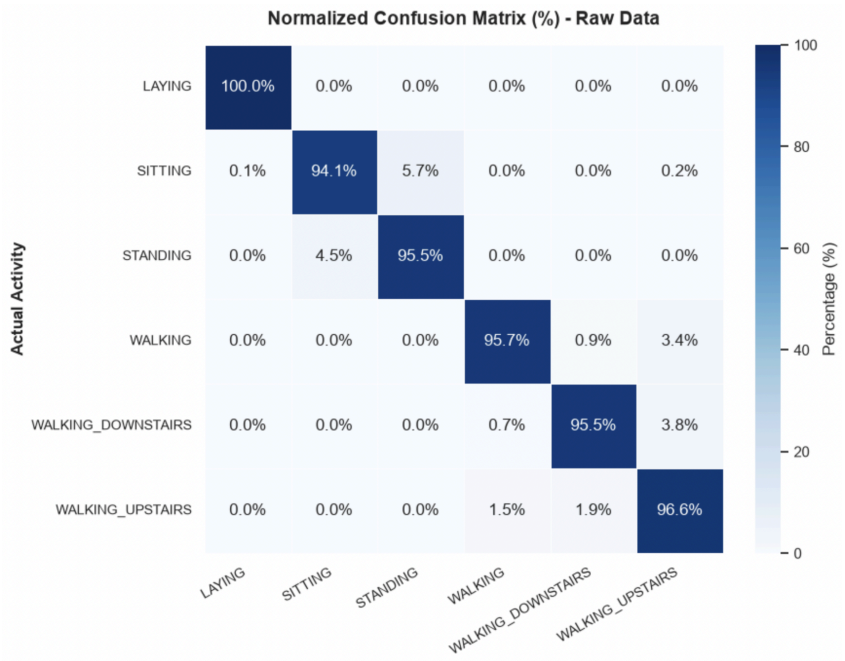
Tishby, N., & Zaslavsky, N. (2015). Deep learning and the information bottleneck principle. 2015 IEEE Information Theory Workshop (ITW), 1–5. https://doi.org/10.1109/itw.2015.7133169

XGBoost Contributors. (2026). Python package introduction: Early stopping. XGBoost Documentation. Retrieved October 7, 2026, from https://xgboost.readthedocs.io/en/stable/python/python_intro.html#early-stopping

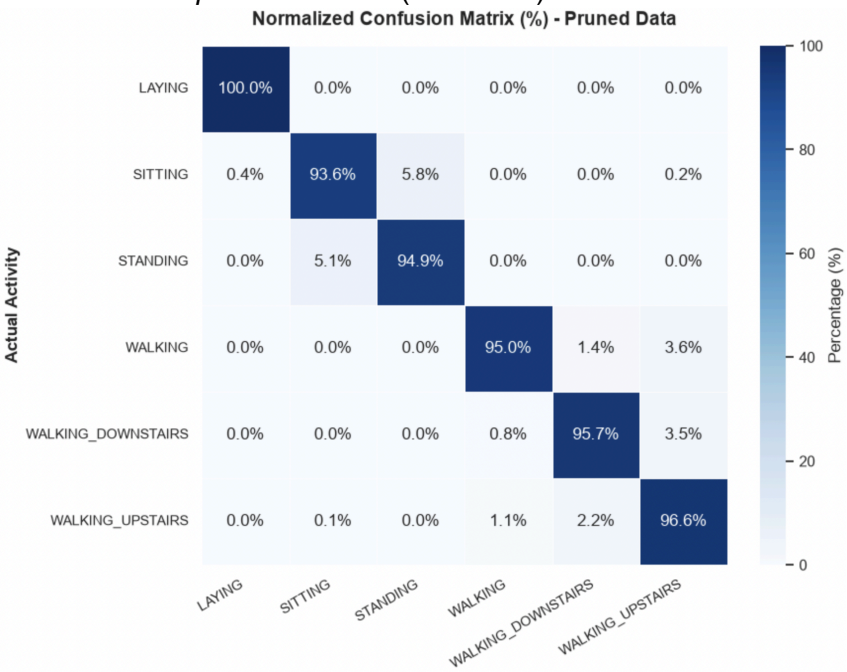
Zhang, W., Zhao, X., & Li, Z. (2019). A comprehensive study of smartphone-based indoor activity recognition via XGBoost. IEEE Access, 7, 80027–80042. https://doi.org/10.1109/access.2019.2922974

Appendix

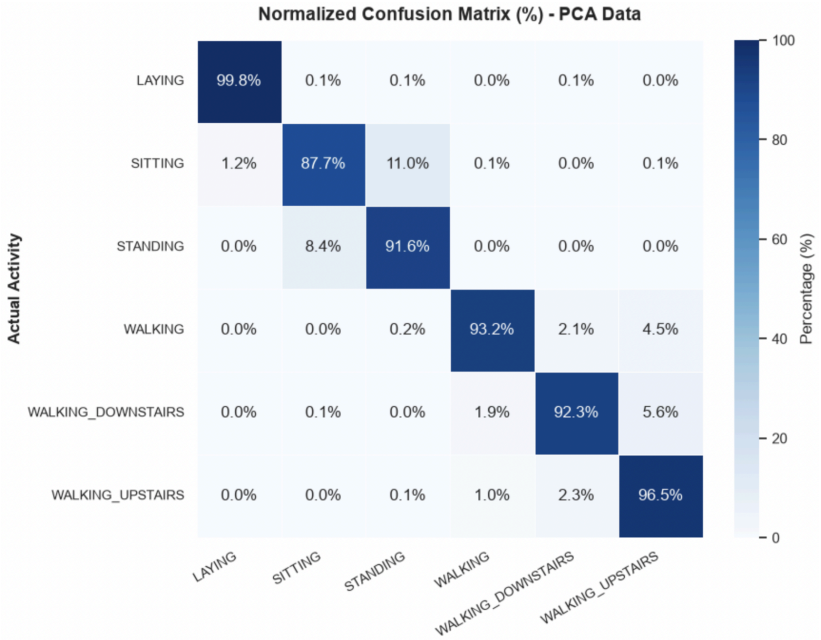
XGBoost confusion matrix for *raw* dataset (Dataset 1)



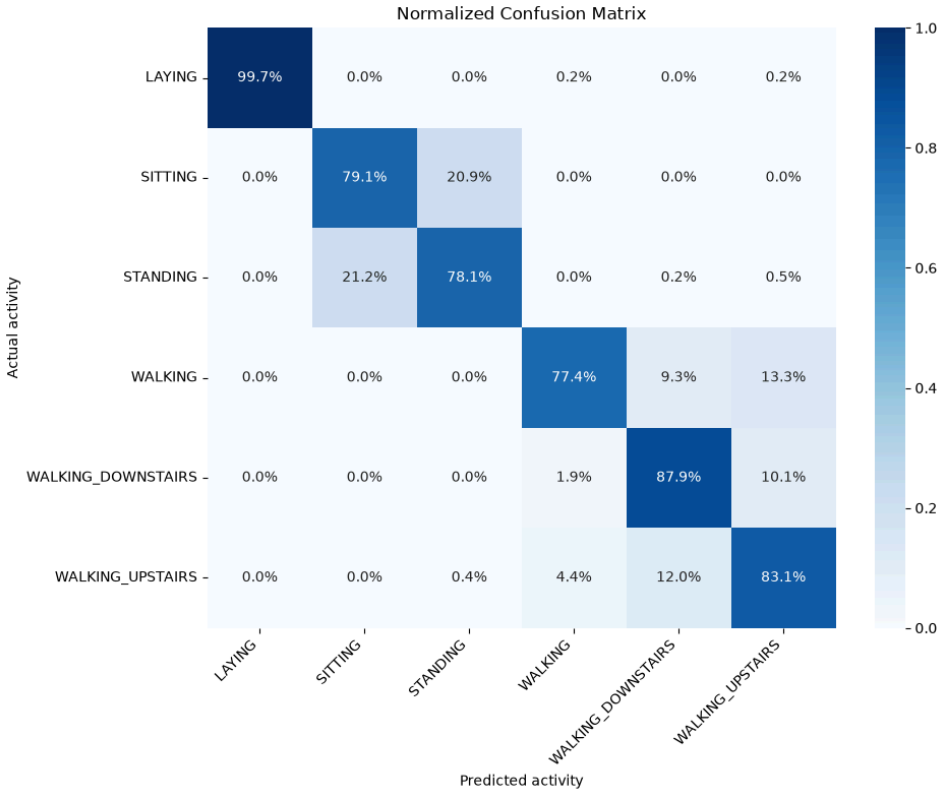
XGBoost confusion matrix for *pruned* dataset (Dataset 2)



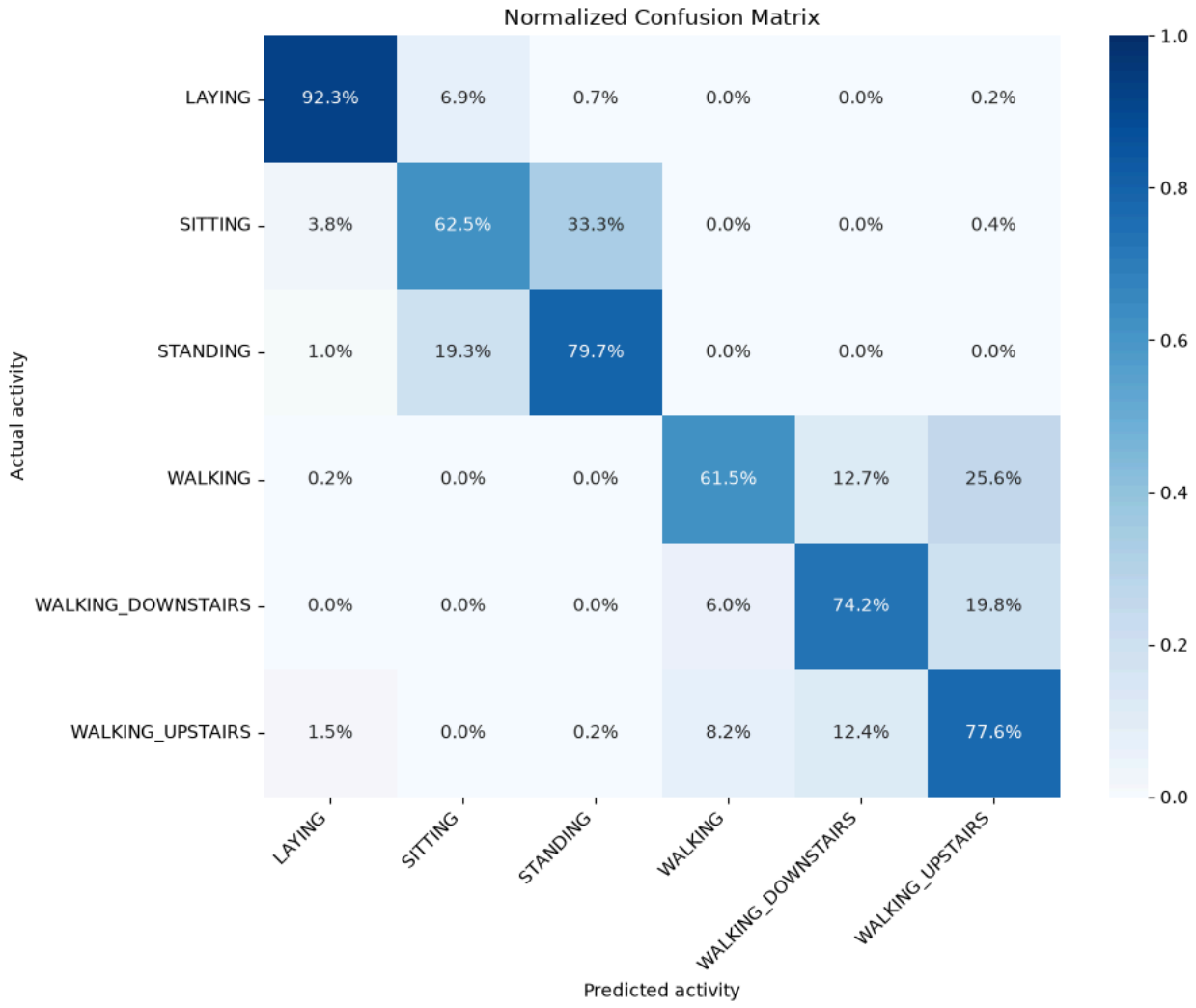
XGBoost confusion matrix for *PCA* dataset (Dataset 3)



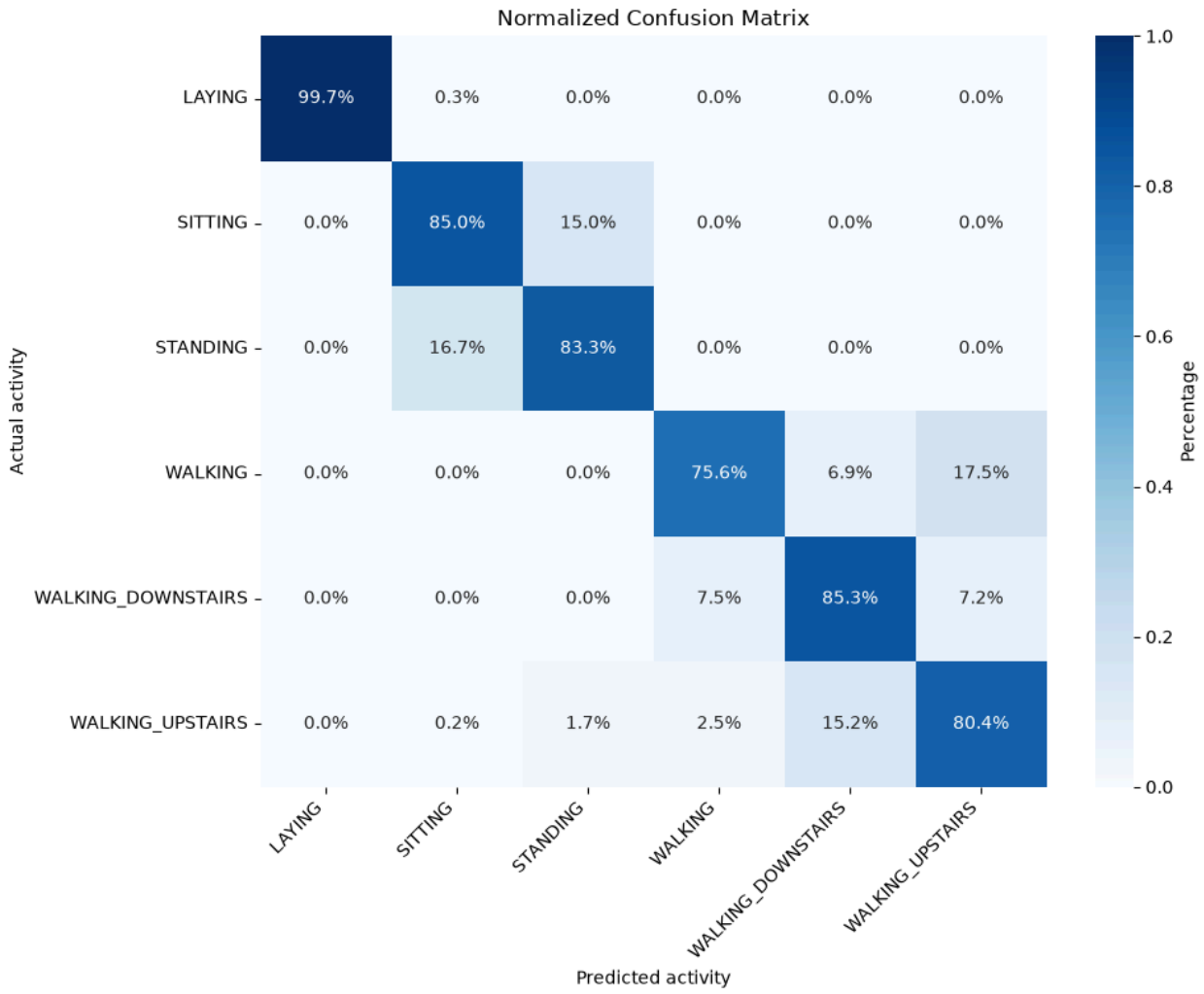
C5.0 confusion matrix for *full* dataset



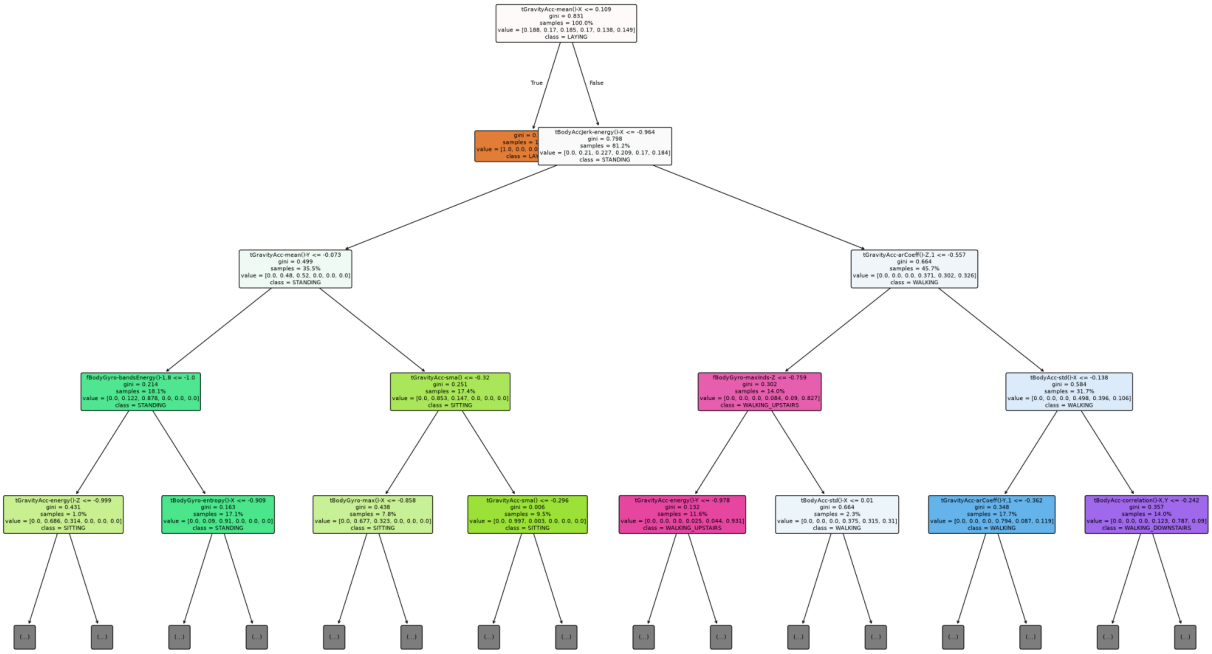
C5.0 confusion matrix for *PCA* dataset



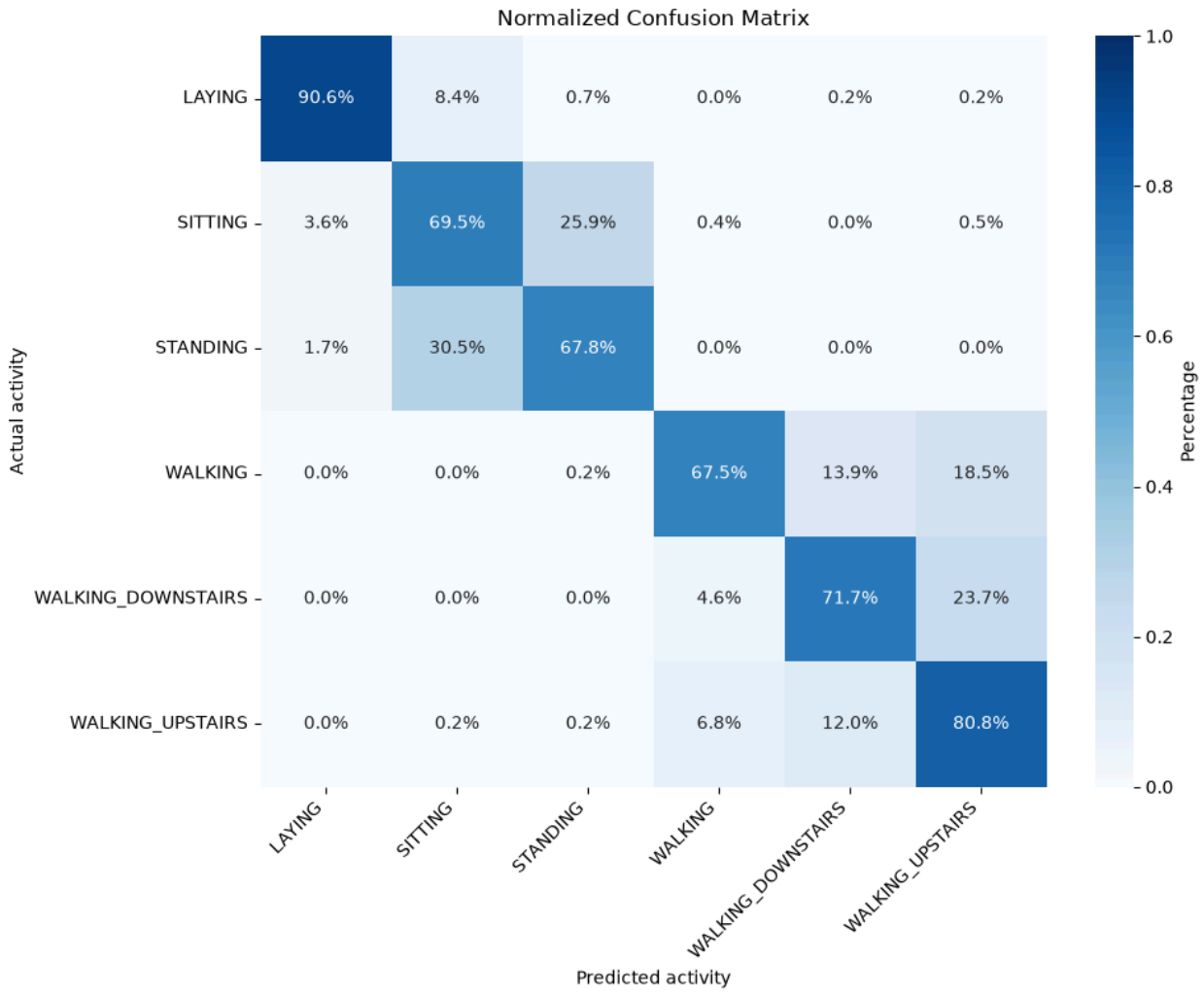
CART confusion matrix for *full* dataset



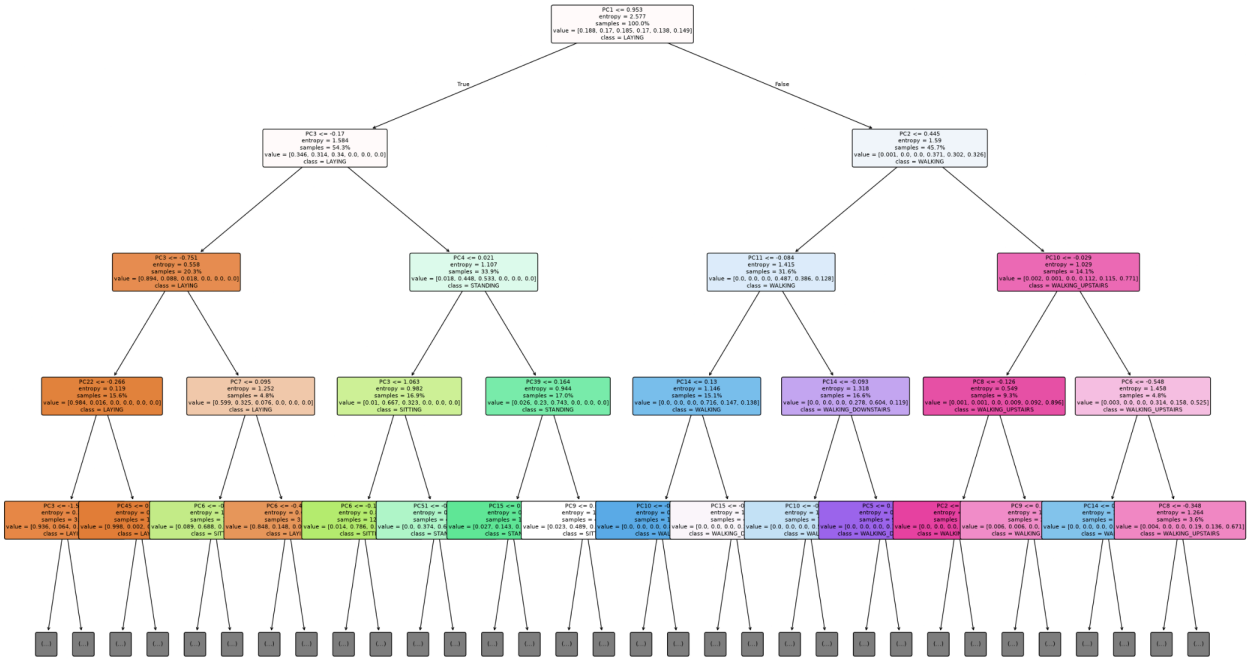
CART tree plot for *full* dataset



CART confusion matrix for *PCA* dataset



CART tree plot for *PCA* dataset


PC1 <= 0.953
entropy = 2.577
samples = 100.0%
value = [0.188, 0.17, 0.185, 0.17, 0.138, 0.149]
class = LAYING
True
False
PC3 <= -0.17
entropy = 1.584
samples = 54.3%
value = [0.346, 0.314, 0.34, 0.0, 0.0, 0.0]
class = LAYING
PC2 <= 0.445
entropy = 1.59
samples = 45.7%
value = [0.001, 0.0, 0.0, 0.371, 0.302, 0.326]
class = WALKING
PC3 <= -0.753
entropy = 0.558
samples = 20.3%
value = [0.894, 0.088, 0.018, 0.0, 0.0, 0.0]
class = LAYING
PC4 <= 0.021
entropy = 1.107
samples = 33.9%
value = [0.018, 0.448, 0.533, 0.0, 0.0, 0.0]
class = STANDING
PC11 <= -0.084
entropy = 1.415
samples = 31.6%
value = [0.0, 0.0, 0.0, 0.487, 0.386, 0.128]
class = WALKING
PC10 <= -0.029
entropy = 1.029
samples = 14.1%
value = [0.002, 0.001, 0.0, 0.112, 0.115, 0.771]
class = WALKING_UPSTAIRS
PC22 <= -0.266
entropy = 0.119
samples = 15.4%
value = [0.984, 0.016, 0.0, 0.0, 0.0, 0.0]
class = LAYING
PC7 <= 0.095
entropy = 1.252
samples = 4.8%
value = [0.599, 0.325, 0.076, 0.0, 0.0, 0.0]
class = LAYING
PC3 <= 1.063
entropy = 0.992
samples = 16.9%
value = [0.03, 0.667, 0.323, 0.0, 0.0, 0.0]
class = SITTING
PC39 <= 0.164
entropy = 0.944
samples = 17.0%
value = [0.026, 0.23, 0.743, 0.0, 0.0, 0.0]
class = STANDING
PC14 <= 0.13
entropy = 1.146
samples = 15.1%
value = [0.0, 0.0, 0.0, 0.0, 0.716, 0.147, 0.138]
class = WALKING
PC14 <= -0.093
entropy = 1.318
samples = 16.6%
value = [0.0, 0.0, 0.0, 0.0, 0.278, 0.604, 0.119]
class = WALKING_DOWNSTAIRS
PC8 <= -0.126
entropy = 0.549
samples = 9.3%
value = [0.001, 0.001, 0.0, 0.009, 0.092, 0.896]
class = WALKING_UPSTAIRS
PC8 <= -0.548
entropy = 1.458
samples = 4.8%
value = [0.003, 0.0, 0.0, 0.314, 0.158, 0.525]
class = WALKING_UPSTAIRS